

```
#!/usr/bin/env python3
```

```
"""
```

blob_campaign_v2.py – Campagne thermalisée avec multiprocessing et auto-test

CORRECTIONS PAR RAPPORT À LA VERSION PRÉCÉDENTE (qui donnait acc=0) :

1. Signe du critère MH explicite et testé :

$\log_ratio = (N * (S_new - S_old)) / \tau$

accept si `rng.random() < exp(min(log_ratio, 0))` [Metropolis standard]

2. Vérification unit-test du critère AVANT la campagne

3. Verdict basé sur `acceptance_prod` :

`acc == 0` → "FROZEN – no moves accepted"

`acc < 0.05` → "OVER_FROZEN"

`0.05 <= acc < 0.20` → "COLD"

`0.20 <= acc <= 0.60` → "V93_COMPLIANT" ← cible

`0.60 < acc <= 0.80` → "WARM"

`acc > 0.80` → "NOT_THERMALIZED"

4. Sauvegarde incrémentale : fichier par seed, pas de perte si interruption

5. Multiprocessing : 1 seed par worker, pool de N_CPU workers

6. Mode auto-test : `--selftest` vérifie le critère avant de tout lancer

USAGE

=====

Test rapide (2 min, 1 seed, N=1000) :

```
python blob_campaign_v2.py --selftest
```

Campagne complète :

```
python blob_campaign_v2.py --N-list 3000 5000 10000 --seeds 5 --workers 4
```

Campagne rapide pour debug (burn+prod = 2N au lieu de 5N) :

```
python blob_campaign_v2.py --N-list 3000 --seeds 2 --fast
```

DÉPENDANCES : `numpy`, `scipy`

```
"""
```

```
import numpy as np
```

```
from scipy.spatial import distance_matrix, cKDTree
```

```
from scipy import stats
```

```
from scipy.sparse import csr_matrix, diags
```

```
from scipy.sparse.linalg import eigsh
```

```
import json
```

```
import time
```

```
import argparse
```

```
import sys
```

```
import os
```

```

import traceback
from pathlib import Path
from multiprocessing import Pool, cpu_count
from functools import partial

# =====
# PARAMÈTRES BLOB (corpus v93) – NE PAS MODIFIER SANS RAISON
# =====

KAPPA_0 = 8          # Degré moyen cible
BETA     = 0.30       # Dissipation
TAU      = 0.50       # Température

# =====
# Classe BlobMCMC – MCMC Metropolis-Hastings correct
# =====

class BlobMCMC:
    """
    Blob avec MCMC Metropolis-Hastings correct pour  $\pi_N \propto \exp(N \cdot S / \tau)$ .

    Le critère d'acceptation est dans mcmc_step(), ligne-par-ligne
    documenté pour être facile à auditer.
    """

    def __init__(self, N, seed, verbose=False):
        self.N = N
        self.seed = seed
        self.rng = np.random.default_rng(seed)
        self.verbose = verbose

        # A4 : positions i.i.d. uniformes dans  $[0,1]^3$ 
        self.positions = self.rng.uniform(0, 1, (N, 3))

        # Rips radius
        c3 = 4.0 * np.pi / 3.0
        self.r_rips = 1.5 * (KAPPA_0 / (c3 * N)) ** (1.0/3.0)

        # Construction graphe initial via cKDTree (rapide pour tout N)
        tree = cKDTree(self.positions)
        pairs = tree.query_pairs(r=self.r_rips, output_type='set')
        self.edges = set((min(i,j), max(i,j)) for i,j in pairs)

        # Adjacence
        self.adj = [set() for _ in range(N)]
        for u, v in self.edges:
            self.adj[u].add(v)
            self.adj[v].add(u)

```

```

self.deg = np.array([len(s) for s in self.adj], dtype=np.int32)

# Triangles (cache)
self.n_tri = self._count_triangles()

def _count_triangles(self):
    """Comptage complet.  $O(N * \langle \text{deg} \rangle^2)$ ."""
    total = 0
    for u in range(self.N):
        nu = self.adj[u]
        # Pour éviter triple comptage : seulement  $v, w > u$  dans nu
        nu_sorted = sorted(x for x in nu if x > u)
        for i, v in enumerate(nu_sorted):
            adj_v = self.adj[v]
            for w in nu_sorted[i+1:]:
                if w in adj_v:
                    total += 1
    return total

def _triangles_through(self, u, v):
    """Nombre de triangles passant par l'arête (u,v)."""
    return len(self.adj[u] & self.adj[v])

def D(self):
    mu = float(self.deg.mean())
    return float(self.deg.std()) / mu if mu > 1e-9 else 0.0

def S_sim(self):
    return np.log(1.0 + self.n_tri) - BETA * self.D()

def mcmc_step(self):
    """
    Un pas MCMC. Retourne True si accepté, False sinon.

    CRITÈRE CRUCIAL :
        log_ratio =  $N * (S_{\text{new}} - S_{\text{old}}) / \tau$ 
        accept si log_ratio  $\geq 0$  (S augmente : accepte toujours)
        sinon accept avec proba  $\exp(\log\_ratio)$ 
    """
    if not self.edges:
        return False

    # 1. Proposer un swap : retirer (u_rm, v_rm), ajouter (u_add, v_add)
    edges_list = list(self.edges)
    idx = self.rng.integers(len(edges_list))
    u_rm, v_rm = edges_list[idx]

```

```

u_add, v_add = -1, -1
for _ in range(100):
    a = int(self.rng.integers(self.N))
    b = int(self.rng.integers(self.N))
    if a != b and b not in self.adj[a]:
        u_add, v_add = (min(a,b), max(a,b))
        if (u_add, v_add) != (u_rm, v_rm):
            break
if u_add == -1:
    return False

# 2. Score avant
S_old = self.S_sim()
n_tri_old = self.n_tri

# 3. Effectuer le swap
tri_rm = self._triangles_through(u_rm, v_rm)
self.adj[u_rm].discard(v_rm); self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1; self.deg[v_rm] -= 1
self.edges.discard((u_rm, v_rm))

tri_add = self._triangles_through(u_add, v_add)
self.adj[u_add].add(v_add); self.adj[v_add].add(u_add)
self.deg[u_add] += 1; self.deg[v_add] += 1
self.edges.add((u_add, v_add))

self.n_tri = n_tri_old - tri_rm + tri_add

# 4. Sanity check (triangles négatifs impossible)
if self.n_tri < 0:
    self._revert(u_rm, v_rm, u_add, v_add, n_tri_old)
    return False

S_new = self.S_sim()

# 5. CRITÈRE METROPOLIS — LE POINT CRITIQUE
# -----
# On veut échantillonner  $\pi_N(C) \propto \exp(N * S(C) / \tau)$ 
# Donc  $P_{\text{accept}} = \min(1, \pi_N(C_{\text{new}}) / \pi_N(C_{\text{old}}))$ 
#  $= \min(1, \exp(N * (S_{\text{new}} - S_{\text{old}}) / \tau))$ 
#
# Soit  $\log\_ratio = N * (S_{\text{new}} - S_{\text{old}}) / \tau$ 
# - Si  $S_{\text{new}} > S_{\text{old}}$  ( $\log\_ratio > 0$ ) : accepte toujours
# - Sinon accepte avec proba  $\exp(\log\_ratio)$ 
# -----
dS = S_new - S_old
log_ratio = self.N * dS / TAU

```

```

if log_ratio >= 0:
    # S augmente : accepte TOUJOURS (bas risque d'erreur ici)
    accepted = True
else:
    # S diminue : Metropolis accepte avec proba exp(log_ratio)
    # log_ratio est négatif, donc exp(log_ratio) ∈ (0, 1)
    if log_ratio < -700: # exp(-700) ≈ 0
        accepted = False
    else:
        accepted = self.rng.random() < np.exp(log_ratio)

if self.verbose:
    print(f"    dS={dS:+.4e}, log_ratio={log_ratio:+.3f}, "
          f"acc={accepted}", flush=True)

if accepted:
    return True
else:
    self._revert(u_rm, v_rm, u_add, v_add, n_tri_old)
    return False

def _revert(self, u_rm, v_rm, u_add, v_add, n_tri_old):
    """Annule le swap."""
    self.adj[u_add].discard(v_add); self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1; self.deg[v_add] -= 1
    self.edges.discard((u_add, v_add))
    self.adj[u_rm].add(v_rm); self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1; self.deg[v_rm] += 1
    self.edges.add((u_rm, v_rm))
    self.n_tri = n_tri_old

# — Mesures spectrales (optionnelles) —————
def compute_lambda1(self):
    """Premier eigenvalue non-nul du Laplacien normalisé."""
    try:
        N = self.N
        row, col = [], []
        for u, v in self.edges:
            row.extend([u, v])
            col.extend([v, u])
        data = np.ones(len(row))
        A = csr_matrix((data, (row, col)), shape=(N, N))
        d = np.array(A.sum(axis=1)).flatten()
        d_safe = np.where(d > 0, d, 1)
        D_inv_sqrt = diags(1.0 / np.sqrt(d_safe))
        L = diags(np.ones(N)) - D_inv_sqrt @ A @ D_inv_sqrt

```

```

        # Trouver les 3 plus petites valeurs propres
        vals, _ = eigsh(L, k=3, which='SM', tol=1e-4, maxiter=1000)
        vals = np.sort(vals)
        # La première non-nulle
        for v in vals:
            if v > 1e-6:
                return float(v)
        return float(vals[-1])
    except Exception as e:
        if self.verbose:
            print(f"    lambda1 failed: {e}")
        return None

# =====
# AUTO-TEST : vérifie le critère MH
# =====

def selftest():
    """
    Lance une simulation N=1000 courte avec logs verbeux pour vérifier :
        - le critère Metropolis donne bien acceptance  $\approx$  20-60%
        - les mesures S/N, n_tri bougent raisonnablement
    """
    print("="*70)
    print("  AUTO-TEST : vérification du critère Metropolis")
    print("="*70)
    print(f"  Paramètres :  $\kappa_0$ ={KAPPA_0},  $\beta$ ={BETA},  $\tau$ ={TAU}")
    print(f"  Test : N=1000, 1 seed, 2000 pas (burn 1000 + prod 1000)\n")

    G = BlobMCMC(N=1000, seed=0, verbose=False)
    print(f"  Graphe initial : |E|={len(G.edges)}, n_tri={G.n_tri}, "
          f"  f\"S/N={G.S_sim()/G.N:.6e}, D={G.D():.4f}")

    # Mini-test manuel : effectuer 10 pas en mode verbose
    print("\n  [Audit] 10 premiers pas MCMC en mode verbose :")
    G.verbose = True
    for i in range(10):
        G.mcmc_step()
    G.verbose = False

    # Burn-in 1000
    print(f"\n  [Burn-in] 1000 pas...")
    t0 = time.time()
    acc_burn = 0
    for _ in range(1000):
        if G.mcmc_step():
            acc_burn += 1

```

```

acc_burn_rate = acc_burn / 1000
print(f" Burn-in done. acc = {acc_burn_rate:.3f} ({acc_burn}/1000) "
      f"in {time.time()-t0:.1f}s")
print(f" État : n_tri={G.n_tri}, S/N={G.S_sim()/G.N:.6e}")

# Production 1000
print(f"\n [Production] 1000 pas...")
t0 = time.time()
acc_prod = 0
S_trace = []
for i in range(1000):
    if G.mcmc_step():
        acc_prod += 1
    if i % 20 == 0:
        S_trace.append(G.S_sim())
acc_prod_rate = acc_prod / 1000
print(f" Production done. acc = {acc_prod_rate:.3f} in {time.time()-t0:.1f}s")
print(f" État final : n_tri={G.n_tri}, S/N={G.S_sim()/G.N:.6e}, "
      f"D={G.D():.4f}")
print(f" S_trace stats : mean={np.mean(S_trace):.4f}, "
      f"std={np.std(S_trace):.4f}")

# Verdict
print("\n" + "="*70)
print(" VERDICT AUTO-TEST")
print("="*70)
if acc_prod_rate == 0.0:
    print(" ❌ FROZEN : aucun pas accepté. Bug dans le critère MH.")
    print("      Inspecter la section '5. CRITÈRE METROPOLIS' du code.")
    return False
elif acc_prod_rate < 0.05:
    print(f" ⚠️ OVER_FROZEN : acc = {acc_prod_rate:.3f} < 0.05")
    print("      τ peut-être trop bas, ou N trop grand pour τ=0.50 à N=1000.")
    print("      Campagne possible mais mélange très lent.")
    return False
elif acc_prod_rate > 0.90:
    print(f" ⚠️ NOT_THERMALIZED : acc = {acc_prod_rate:.3f} > 0.90")
    print("      Facteur N probablement absent ou mal appliqué.")
    return False
elif 0.10 <= acc_prod_rate <= 0.80:
    print(f" ✅ ACCEPTABLE : acc = {acc_prod_rate:.3f} ∈ [0.10, 0.80]")
    print("      Critère MH opérationnel.")
    if 0.20 <= acc_prod_rate <= 0.60:
        print(f" ✅ V93_COMPLIANT : acc ∈ [0.20, 0.60]. Optimal.")
    return True
else:
    print(f" ⚠️ Acc = {acc_prod_rate:.3f} hors plage optimale.")

```

```

    return False

# =====
# Worker pour une seed (appelé par multiprocessing.Pool)
# =====
def run_one_seed(task):
    """
    Exécute une seed complète. task = dict(N, seed, burn_in, production,
    output_dir, compute_lambda).
    """
    N = task['N']
    seed = task['seed']
    burn_in = task['burn_in']
    production = task['production']
    output_dir = task['output_dir']
    compute_lambda = task.get('compute_lambda', True)

    t_start = time.time()
    log_prefix = f"[N={N}|seed={seed}|pid={os.getpid()}]"
    print(f"{log_prefix} START", flush=True)

    try:
        G = BlobMCMC(N, seed)
        print(f"{log_prefix} Init done: |E|={len(G.edges)}, n_tri_init={G.n_tri}"
              flush=True)

        # — Burn-in —
        acc_burn = 0
        t0 = time.time()
        log_every = max(1, burn_in // 10)
        for step in range(burn_in):
            if G.mcmc_step():
                acc_burn += 1
            if (step + 1) % log_every == 0:
                elapsed = time.time() - t0
                eta = elapsed * (burn_in - step - 1) / (step + 1)
                acc_cur = acc_burn / (step + 1)
                print(f"{log_prefix} burn {step+1}/{burn_in} "
                      f"acc={acc_cur:.3f} S/N={G.S_sim()/N:.4e} "
                      f"elapsed={elapsed:.0f}s eta={eta:.0f}s", flush=True)

        acc_rate_burn = acc_burn / burn_in if burn_in > 0 else 0.0
        print(f"{log_prefix} Burn-in done. acc={acc_rate_burn:.3f}", flush=True)

        # — Production —
        acc_prod = 0

```

```

S_trace = []
D_trace = []
ntri_trace = []
sample_every = max(1, production // 500)
t0 = time.time()

for step in range(production):
    if G.mcmc_step():
        acc_prod += 1
    if step % sample_every == 0:
        S_trace.append(G.S_sim())
        D_trace.append(G.D())
        ntri_trace.append(G.n_tri)
    if (step + 1) % log_every == 0:
        elapsed = time.time() - t0
        eta = elapsed * (production - step - 1) / (step + 1)
        acc_cur = acc_prod / (step + 1)
        print(f"{log_prefix} prod {step+1}/{production} "
              f"acc={acc_cur:.3f} S/N={G.S_sim()/N:.4e} "
              f"elapsed={elapsed:.0f}s eta={eta:.0f}s", flush=True)

acc_rate_prod = acc_prod / production if production > 0 else 0.0
S_arr = np.array(S_trace)
D_arr = np.array(D_trace)
ntri_arr = np.array(ntri_trace)

# — Mesures spectrales —
lambda1 = None
if compute_lambda:
    print(f"{log_prefix} Computing  $\lambda_1$ ...", flush=True)
    t_lam = time.time()
    lambda1 = G.compute_lambda1()
    print(f"{log_prefix}  $\lambda_1$  = {lambda1} (in {time.time()-t_lam:.1f}s)",
          flush=True)

# — Verdict —
if acc_rate_prod == 0.0:
    verdict = "FROZEN_BUG"
elif acc_rate_prod < 0.05:
    verdict = "OVER_FROZEN"
elif acc_rate_prod < 0.20:
    verdict = "COLD"
elif acc_rate_prod <= 0.60:
    verdict = "V93_COMPLIANT"
elif acc_rate_prod <= 0.80:
    verdict = "WARM"
else:

```

```

        verdict = "NOT_THERMALIZED"

result = {
    'N': int(N),
    'seed': int(seed),
    'burn_in': int(burn_in),
    'production': int(production),
    'kappa_0': KAPPA_0, 'beta': BETA, 'tau': TAU,
    'acc_rate_burn': float(acc_rate_burn),
    'acc_rate_prod': float(acc_rate_prod),
    'S_mean': float(S_arr.mean()),
    'S_std': float(S_arr.std()),
    'S_per_N_mean': float(S_arr.mean() / N),
    'S_per_N_std': float(S_arr.std() / N),
    'D_mean': float(D_arr.mean()),
    'D_std': float(D_arr.std()),
    'ntri_mean': float(ntri_arr.mean()),
    'ntri_std': float(ntri_arr.std()),
    'lambda_1': float(lambdal) if lambdal else None,
    'n_samples': int(len(S_trace)),
    'total_time_sec': float(time.time() - t_start),
    'verdict': verdict
}

# Sauvegarde immédiate
out_path = Path(output_dir) / f"blob_N{N}_seed{seed}.json"
with open(out_path, 'w') as f:
    json.dump(result, f, indent=2)

print(f"{log_prefix} DONE. acc={acc_rate_prod:.3f} "
      f"S/N={result['S_per_N_mean']:.4e} verdict={verdict} "
      f"time={result['total_time_sec']:.0f}s", flush=True)

return result

except Exception as e:
    print(f"{log_prefix} ERROR: {e}", flush=True)
    traceback.print_exc()
    return {'N': N, 'seed': seed, 'error': str(e)}

# =====
# Main
# =====
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--selftest', action='store_true',

```

```

        help='Lancer uniquement 1\'auto-test (2 min)')
parser.add_argument('--N-list', type=int, nargs='+', default=[3000, 5000, 100
        help='Tailles à simuler')
parser.add_argument('--seeds', type=int, default=5, help='Seeds par taille')
parser.add_argument('--burn-factor', type=float, default=5.0,
        help='Burn-in = burn-factor * N')
parser.add_argument('--prod-factor', type=float, default=5.0,
        help='Production = prod-factor * N')
parser.add_argument('--workers', type=int, default=None,
        help='Nombre de processus parallèles (défaut: CPU count)')
parser.add_argument('--output-dir', type=str, default='.', help='Output dir')
parser.add_argument('--fast', action='store_true',
        help='Mode rapide : burn/prod = 2N au lieu de 5N')
parser.add_argument('--no-lambda', action='store_true',
        help='Ne pas calculer  $\lambda_1$  (gagne du temps)')
args = parser.parse_args()

# — Auto-test —
if args.selftest:
    ok = selftest()
    return 0 if ok else 1

# — Mode fast —
if args.fast:
    args.burn_factor = 2.0
    args.prod_factor = 2.0
    print("⚠️  MODE FAST : burn/prod = 2N (non v93-compliant)")

# — Configuration —
workers = args.workers or min(cpu_count(), args.seeds * len(args.N_list))
Path(args.output_dir).mkdir(parents=True, exist_ok=True)

print("="*70)
print("  BLOB THERMALIZED CAMPAIGN v2 – multiprocessing")
print("="*70)
print(f"  N list          : {args.N_list}")
print(f"  Seeds per N     : {args.seeds}")
print(f"  Burn-in         : {args.burn_factor} × N")
print(f"  Production      : {args.prod_factor} × N")
print(f"  Workers         : {workers} (cpu_count={cpu_count()})")
print(f"  Output          : {args.output_dir}")
print(f"  Compute  $\lambda_1$  : {not args.no_lambda}")
print("="*70)

# — Construction des tâches —
tasks = []
for N in args.N_list:

```

```

    burn = int(args.burn_factor * N)
    prod = int(args.prod_factor * N)
    for seed in range(args.seeds):
        tasks.append({
            'N': N, 'seed': seed,
            'burn_in': burn, 'production': prod,
            'output_dir': args.output_dir,
            'compute_lambda': not args.no_lambda
        })

print(f"\n Total tasks : {len(tasks)} "
      f"({len(args.N_list)} sizes × {args.seeds} seeds)")

# — Lancement parallèle —
t_campaign = time.time()
print(f"\n Lancement avec {workers} workers...\n")

with Pool(processes=workers) as pool:
    all_results = pool.map(run_one_seed, tasks)

total_time = time.time() - t_campaign

# — Agrégation —
print("\n" + "="*70)
print(f" CAMPAIGN DONE : {total_time/60:.1f} min ({total_time/3600:.2f}h)")
print("="*70)

# Filtrer succès
ok_results = [r for r in all_results if 'error' not in r]
fail_results = [r for r in all_results if 'error' in r]
print(f" Successful : {len(ok_results)}/{len(tasks)}")
if fail_results:
    print(f" Failed : {len(fail_results)}")
    for r in fail_results:
        print(f" N={r['N']} seed={r['seed']} : {r['error']}")

# Résumé par N
by_N = {}
for r in ok_results:
    by_N.setdefault(r['N'], []).append(r)

summary = {
    'campaign_time_sec': float(total_time),
    'N_list': args.N_list,
    'seeds': args.seeds,
    'workers': workers,
    'burn_factor': args.burn_factor,

```

```

        'prod_factor': args.prod_factor,
        'params': {'kappa_0': KAPPA_0, 'beta': BETA, 'tau': TAU},
        'by_N': {}
    }

print("\n Résumé par N :")
print(f"  {'N':<8}{'acc_prod':<12}{'S/N':<16}{'n_tri':<14}{'λ_1':<10}{'verdic
print(f"  {'-'*80}")
for N in sorted(by_N.keys()):
    rs = by_N[N]
    acc = np.mean([r['acc_rate_prod'] for r in rs])
    SN = np.mean([r['S_per_N_mean'] for r in rs])
    SN_std = np.std([r['S_per_N_mean'] for r in rs])
    ntri = np.mean([r['ntri_mean'] for r in rs])
    lam_vals = [r['lambda_1'] for r in rs if r['lambda_1']]
    lam = np.mean(lam_vals) if lam_vals else None
    verdicts = [r['verdict'] for r in rs]
    # Verdict majoritaire
    from collections import Counter
    verdict_main = Counter(verdicts).most_common(1)[0][0]

    summary['by_N'][str(N)] = {
        'n_seeds_ok': len(rs),
        'acc_prod_mean': float(acc),
        'S_per_N_mean': float(SN),
        'S_per_N_std': float(SN_std),
        'ntri_mean': float(ntri),
        'lambda_1_mean': float(lam) if lam else None,
        'verdict_majority': verdict_main,
        'all_verdicts': verdicts
    }

    lam_str = f"{lam:.4f}" if lam else "N/A"
    print(f"  {N:<8}{acc:<12.3f}{SN:<16.4e}{ntri:<14.1f}{lam_str:<10}{verdict

# Loi d'échelle
Ns_fit = sorted(by_N.keys())
if len(Ns_fit) >= 2:
    SN_fit = [summary['by_N'][str(N)]['S_per_N_mean'] for N in Ns_fit]
    if all(x > 0 for x in SN_fit):
        log_N = np.log(Ns_fit)
        log_SN = np.log(SN_fit)
        slope, intercept, r_val, _, _ = stats.linregress(log_N, log_SN)
        summary['scaling_fit'] = {
            'b_exponent': float(-slope), # S/N ~ N^-b, on veut b positif
            'a_prefactor': float(np.exp(intercept)),
            'R2': float(r_val**2),
            'v93_b': 0.870,

```

```

        'B3_b': 1.000
    }
    print(f"\n Loi d'échelle : S/N = {np.exp(intercept):.3f} × N^{slope:}
    print(f"      R² = {r_val**2:.4f}")
    print(f"      b mesuré   = {-slope:.3f}")
    print(f"      b v93      = 0.870")
    print(f"      b B3       = 1.000")
    if abs(-slope - 0.870) < abs(-slope - 1.000):
        print(f"      → COMPATIBLE v93")
    else:
        print(f"      → COMPATIBLE B3")

# Sauvegarde summary
with open(Path(args.output_dir) / "blob_campaign_summary.json", 'w') as f:
    json.dump(summary, f, indent=2)
print(f"\n Summary saved: blob_campaign_summary.json")

return 0

if __name__ == "__main__":
    sys.exit(main())

```